



## 第九章 排序

杨震 计算机学院

## 9.1 基本概念与术语

BUPT

**排序**是根据记录关键字的值的**递增(递减)**的关系将文件记录的次序重新排列。

### [定义]

设有含 $n$ 个记录的序列  $\{R_1, R_2, \dots, R_n\}$ ,  
对应的关键字序列为  $\{K_1, K_2, \dots, K_n\}$ ,  
一个置换  $p_1, p_2, \dots, p_n$   
使记录序列按关键字有序  $\{R_{p_1}, R_{p_2}, \dots, R_{p_n}\}$ ,  
满足  $K_{p_1} \leq K_{p_2} \leq \dots \leq K_{p_n}$  (**正序**)  
或  $K_{p_1} \geq K_{p_2} \geq \dots \geq K_{p_n}$  (**逆序**)

## 排序的分类

### [1. 根据排序时文件记录的存放位置]

**内部排序**：排序过程中将全部记录放在内存中处理。

**外部排序**：排序过程中需在内外存之间交换信息。

### [2. 根据排序前后相同关键字记录的相对次序]

**稳定排序**：设文件中任意两个记录的关键字值相同，即  $K_i = K_j (i \neq j)$ ，若排序之前记录  $R_i$  领先于记录  $R_j$ ，排序后这种关系不变(对所有输入实例而言)。

**不稳定排序**：只要有一个实例使排序算法不满足稳定性要求。

### [3. 根据文件的存储结构划分排序的种类]

连续顺序文件排序

链表排序

地址排序: 待排记录顺序存储, 排序时只对辅助表(关键字+指针)的表目进行物理重排。

### [4. 根据排序的方法]

插入排序

交换排序

选择排序

归并排序

基数排序

### [5. 根据排序算法所需的辅助空间]

就地排序:  $O(1)$  非就地排序:  $O(n)$ 或与 $n$ 有关

## 评价排序算法的主要标准

### [时间开销]

考察算法的两个基本操作的次数：

- 比较关键字
- 移动记录

算法时间还与输入实例的初始状态有关时，分情况：

- 最好
- 最坏
- 平均

### [空间开销]

所需的辅助空间

## 约定

顺序方式存储时，结构如下

**# define** MAXSIZE 最大记录个数

**typedef int** KeyType; 关键字类型如果是整型

**typedef struct** {

    KeyType key;

    InfoType otherinfo;

} ElemType;

## 9.2 插入排序

BUPT

### 直接插入排序（增量法）

**[ 示例 ]** { R(0) R(-4) R(8) R(1) R(-4) R(-6) } n=6

初态 [ 0 ] -4 8 1 -4 -6 把第一个记录看成有序序列

i=2 [ -4 0 ] 8 1 -4 -6

i=3 [ -4 0 8 ] 1 -4 -6 稳定的排序算法

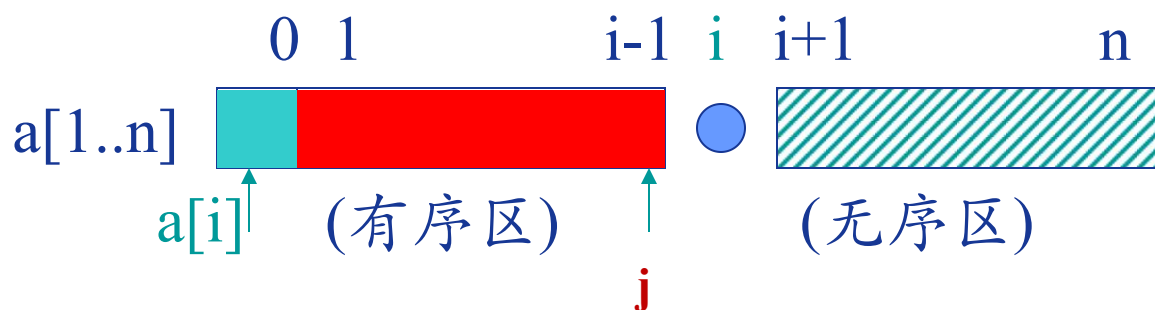
i=4 [ -4 0 1 8 ] -4 -6

i=5 [ -4 -4 0 1 8 ] -6

i=6 [ -6 -4 -4 0 1 8 ]

**[算法思想]** 每次使有序区增加一个记录

## [算法步骤]



循环 $(n-1)$ 次，初值  $i=2$ ， $a[0]$  用作哨兵。

1) 把第  $i$  个记录取出保存在  $a[0]$  中

2) 若  $a[0].key < a[i].key$ ，则  $a[i]$  后移一位， $i--$ ，转2)；

否则  $a[0]$  放在  $a[i+1]$  处，转1)



### [一趟插入过程]

```
void insert(ElemType e, Elemtype a[], int i)
//a[1..i]有序，插入e
{
    a[0] = e;
    while (e.key < a[i].key)
    {
        a[i+1] = a[i];
        i--;
    }
    a[i+1] = e;
}
```

### [插入排序]

```
void InsertSort(ElemType a[], int n)
//对a[1..n]排序
{
    int j;
    ElemType temp;
    for (j=2; j<= n; j++)
    {
        temp = a[j];
        insert(temp, a, j-1);
    }
}
```

## [哨兵/监视哨的作用]

简化边界条件的测试，提高算法时间效率。

## [性能分析]

❖ 最好情况(原始数据按**正序**即非递减序排列)

$$C_{\min}=n-1 \quad M_{\min}=2(n-1)$$

❖ 最坏情况(原始数据按**逆序**即非递增序排列)

$$C_{\max}=(n+2)(n-1)/2 \quad M_{\max}=(n+4)(n-1)/2$$

❖ 随机情况

$$C_{\text{avg}}=(C_{\min}+C_{\max})/2 \approx n^2/4 \quad M_{\text{avg}} \approx n^2/4$$

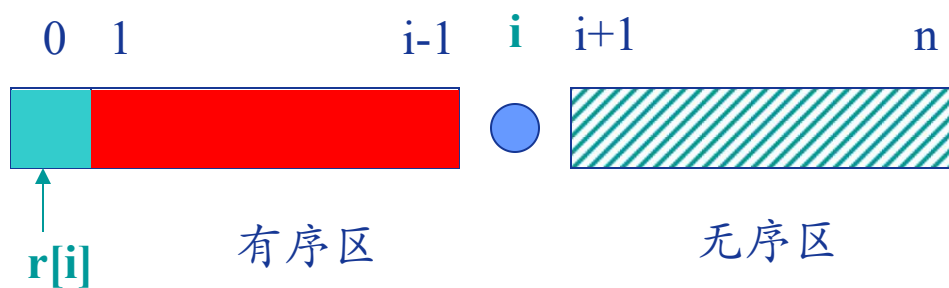
❖ 时间复杂度 $O(n^2)$

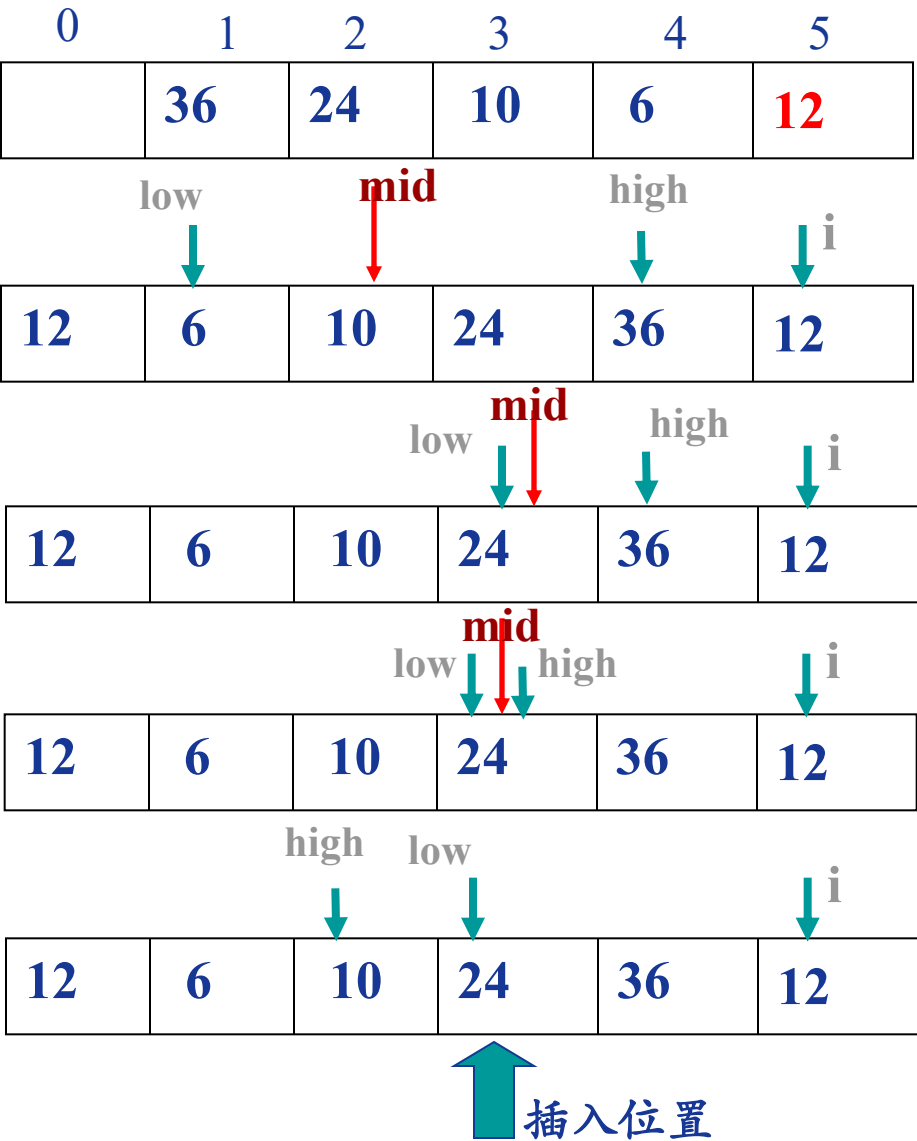
辅助空间复杂度 $O(1)$

## 改进措施1：折半插入排序

算法思想：将循环中每一次在有序区  $[1, i-1]$  上为确定  $r[i]$  插入位置的顺序查找操作改为折半查找操作。

效果：减少关键字间的比较次数。





$mid = \lfloor (low + high) / 2 \rfloor$

## [算法描述]

```
void BInsertSort(ElemType a[], int n)
{
    for ( i = 2; i <= n; ++i )
    {
        a[0] = a[i]; low = 1 ; high = i-1 ;
        while ( low <= high )
        {
            m = ( low + high ) / 2 ;
            if ( a[0].key < a[m].key ) )  high = m -1 ;
            else low = m + 1;
        }
        for ( j=i-1; j>=high+1; - - j )
            a[j+1] = a[j];
        a[high+1] = a[0];
    }
}
```

## 改进措施1： 2-路插入排序

算法思想： 设置与r同样大小的辅助空间d，将**r[1]**赋值给d[1]，将d看作循环向量。对于r[i] ( $2 \leq i \leq n$ )，若 $r[i] \geq d[1]$ ，则插入d[1]之后的有序序列中，反之则插入d[1]之前的有序序列中。(避免**r[1]**关键字最小/最大)

效果： 减少记录的移动次数

## 希尔排序

### [算法思想的出发点]

- ❖ 直接插入排序在待排序列的关键字基本有序时，效率较高
- ❖ 在待排序的记录个数较少时，效率较高

### [算法思想]

先选定一个记录下标的增量 $d$ ，将整个记录序列按增量 $d$ 从第一个记录开始划分为若干组，对每组使用直接插入排序的方法；然后减小增量 $d$ ，不断重复上述过程，如此下去，直到 $d=1$ (此时整个序列是一组)。

### [ 示例 ]

<初态>  
d<sub>1</sub>=5

|    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|
| 46 | 82 | 52 | 40 | 67 | 31 | 40 | 73 | 53 | 76 |
| 31 |    |    |    |    | 46 |    |    |    |    |
|    | 40 |    |    |    |    | 82 |    |    |    |
|    |    | 52 |    |    |    |    | 73 |    |    |
|    |    |    | 40 |    |    |    |    | 53 |    |
|    |    |    |    | 67 |    |    |    |    | 76 |

<第一趟结果>  
d<sub>2</sub>=3

|    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|
| 31 | 40 | 52 | 40 | 67 | 46 | 82 | 73 | 53 | 76 |
| 31 |    |    | 40 |    |    | 76 |    |    | 82 |
|    | 40 |    |    | 67 |    |    | 73 |    |    |
|    |    | 46 |    |    | 52 |    |    | 53 |    |

<第二趟结果>  
d<sub>3</sub>=1

|    |    |    |    |    |    |    |    |    |    |
|----|----|----|----|----|----|----|----|----|----|
| 31 | 40 | 46 | 40 | 67 | 52 | 76 | 73 | 53 | 82 |
| 31 | 40 | 40 | 46 | 52 | 53 | 67 | 73 | 76 | 82 |

不稳定排序

程序实现：类似于直接插入排序的程序。  
注意修改步长。



### [希尔排序算法]

```
void Shellsort(ElmntType a[], int n) //对a[1..n]排序
{
    int i,j,d;
    for (d = n/2; d >0; d /=2) //一组流行但不好的增量 n/2,n/4,n/8....1
        for (i=d+1; i<=n; i++)
        {
            ElmntType tmp = a[i];
            for (j=i; j>=d; j -= d)
                if(tmp.key < a[j-d].key)
                {
                    a[j] = a[j-d];
                }
            else
                break;
            a[j] = tmp;
        }
}
```

## [性能分析]

- ❖ 时间复杂度是 $n$ 和 $d$ 的函数
- ❖ 如何选择最佳 $d$ 序列，目前尚未解决
  - 最后一个增量值必须为1
  - 避免增量序列中的值（尤其是相邻的值）有公因子

| 初态    | 1 | 9 | 2 | 10 | 3 | 11 | 4 | 12 | 5 | 13 | 6  | 14 | 7  | 15 | 8  | 16 |
|-------|---|---|---|----|---|----|---|----|---|----|----|----|----|----|----|----|
| $d=8$ | 1 | 9 | 2 | 10 | 3 | 11 | 4 | 12 | 5 | 13 | 6  | 14 | 7  | 15 | 8  | 16 |
| $d=4$ | 1 | 9 | 2 | 10 | 3 | 11 | 4 | 12 | 5 | 13 | 6  | 14 | 7  | 15 | 8  | 16 |
| $d=2$ | 1 | 9 | 2 | 10 | 3 | 11 | 4 | 12 | 5 | 13 | 6  | 14 | 7  | 15 | 8  | 16 |
| $d=1$ | 1 | 2 | 3 | 4  | 5 | 6  | 7 | 8  | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |

- ❖ 实验结果：当 $n$ 较大时，比较和移动次数约在 $n^{1.25}$ 到 $1.6n^{1.25}$ 。
- ❖ 原地排序

## 9.3 交换排序

BUPT

### 起泡排序(冒泡排序)

#### [算法思想]

将两个相邻记录的关键字进行比较,若为逆序则交换两者位置,小者往上浮,大者往下沉。

#### [算法步骤]

记录1和2、2和3、.....、 $(n-1)$ 和 $n$ 的关键字比较(交换);

记录1和2、2和3、.....、 $(n-2)$ 和 $(n-1)$ 的关键字比较(交换);

.....

直到某一趟不出现交换操作为止。

[ 示例 ]

| <初 态>     | <第一趟>     | <第二趟>     | <第三趟>     | <第四趟>     |
|-----------|-----------|-----------|-----------|-----------|
| 0         | -4        | -4        | -6        | -6        |
| -4        | 0         | -6        | -4        | -4        |
| 8         | -6        | <u>-4</u> | <u>-4</u> | <u>-4</u> |
| -6        | <u>-4</u> | 0         | 0         | 0         |
| <u>-4</u> | 1         | 1         | 1         | 1         |
| 1         | 8         | 8         | 8         | 8         |

稳定排序

sorted

F

F

F

T

## [冒泡排序算法]

```
void BubbleSort(ElemType a[], int n) // 自上而下冒泡
```

```
{  
    int change = 1;  
    low=1;  
    high=n; //冒泡的上下界  
    while(low < high && change)  
    {  
        change=0;          //设不发生交换  
        for(i=low; i<high; i++) //从上向下起泡  
            if(a[i].key > a[i+1].key)  
            {  
                swap(&a[i],&a[i+1]);  
                change=1;  
            }  
        //有交换，修改标志change  
        high--; //修改上界  
    }  
}
```

## [性能分析]

- ❖ 最好情况(原始数据按正序即非递减序排列)

$$C_{\min}=n-1 \quad M_{\min}=0$$

- ❖ 最坏情况(原始数据按逆序即非递增序排列)

$$C_{\max}=n(n-1)/2 \quad M_{\max}=3n(n-1)/2$$

- ❖ 时间复杂度 $O(n^2)$

辅助空间复杂度 $O(1)$

## [算法的改进]

- ❖ 每趟排序中，记录最后一次发生交换的位置
- ❖ 双向交替起泡，下→上，最轻升顶；上→下，最重沉底

**思考：双向起泡算法如何实现？**

```
void BubbleSort2(ElemType a[],int n) //相邻两趟向相反方向起泡的冒泡排序算法
{
    change=1; low=1; high=n; //冒泡的上下界
    while(low < high && change)
    {
        change=0;           //设不发生交换
        for(i=low; i<high; i++) //从上向下起泡
            if(a[i].key>a[i+1].key){
                swap(&a[i],&a[i+1]);
                change=1;
            }
        //有交换，修改标志change
        high--; //修改上界
        for(i=high; i>low; i--) //从下向上起泡
            if(a[i].key<a[i-1].key){
                swap(&a[i], &a[i-1]);
                change=1;
            }
        low++; //修改下界
    }
}
```

思考：如以双链表存储记录，双向起泡算法如何实现？

## 快速排序(分划交换排序/分治法)

### [分治算法原理]

递归使用两个调用，每个调用大约处理一半信息(二叉树遍历，Hanior塔)。

- 1)分解：将原问题分解为若干子问题
- 2)求解：递归地解各子问题，若子问题的规模足够小，则直接求解
- 3)组合：将各子问题的解组合成原问题的解

### [分治法性质]

把大小为n的问题分解为两个独立非空部分的递归函数，它递归调用自身的次数不多于n次。

如果两部分的大小分别为k和n-k，则递归总的调用次数为： $T_n = T_k + T_{n-k} + 1$ ;  
 $T_1 = 0$ ；通过归纳法可得 $T_n = n - 1$ 。

### [快速排序算法思想]

指定**枢轴/支点/基准记录** $r[p]$ (通常为第一个记录)，通过一趟排序将其放在正确的位置上，它把待排记录分割为独立的两部分，使得

左边记录的关键字  $\leq r[p].key \leq$  右边记录的关键字

对左右两部分记录序列重复上述过程，依次类推，直到子序列中只剩下一个记录或不合记录为止。(可以用递归方法实现)



```
tmp=49
```

left ←.....right

27 38 65 97 76 13 ( ) 49

left  $\longrightarrow$  right

27 38 ( ) 97 76 13 65 49

left       $\leftarrow$  right

27 38 13 97 76 ( ) 65 49

left  $\cdots \rightarrow$  right

27 38 13 ( ) 76 97 65 49

left ← ..... right

<一趟结束> {27 38 13} **49** {76 97 65 49}

(left =right)

<二趟结束> {13} 27 {38} 49 {49 65} 76 {97}

<三趟结束> {13} 27 {38} 49 49 {65} 76 {97} //所有子序列长度=0,1结束

```
void QuickSort ( ElemType a[] int n )  
{   QSort ( a, 1, n); } // QuickSort  
    // 对顺序表 a[1..n]进行快速排序
```

```
void QSort ( ElemType a[], int left, int right )  
{   if ( left < right )  
    {   pivotloc = Partition(a, left, right ) ;  
        Qsort (a, left, pivotloc-1) ;  
        Qsort (a, pivotloc+1, right);  
    }  
} // QSort
```

## [一趟划分]

```
int Partition (ElemType a[], int left, int right) //返回枢纽记录的下标
{
    Elemtype tmp = a[left]; pivotkey = a[left].key;
    while ( left < right )
    {
        while ( left < right && a[right].key > pivotkey ) --right;
        a[left] = a[right];
        while ( left < right && a[low].key < pivotkey ) ++left;
        a[right] = a[left];
    }
    a[left]= tmp;
    return left;
} // Partition
```

[性能分析]

最坏情况(原始数据正/逆序排列)

$C_{\max}=n(n-1)/2$        $M_{\max} \leq C_{\max}$        $O(n^2)$   
最好情况：每次划分的结果是基准的左、右两个无序子区间的长度大致相等

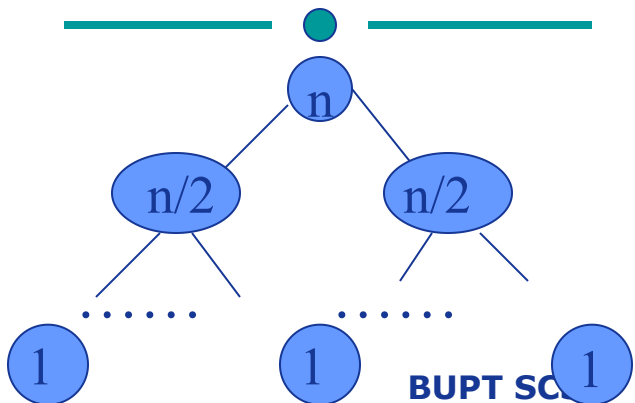
$C_{\min} \leq O(n \log_2 n)$        $M_{\min} \leq C_{\min}$        $O(n \log_2 n)$   
平均时间性能       $T_{\text{avg}}(n)=kn \log_2 n$

k: 某个常数; n: 待排序序列中记录个数

就平均时间而言，快速排序是目前被认为最好的一种内部排序方法。

辅助空间复杂度    最好情况  $O(\log_2 n)$   
最坏情况  $O(n)$

快速排序的最大递归深度是多少？最小递归深度是多少？



## [快速排序的非递归算法]

```

#define push2(A,B) push(A); push(B);
void QSort ( ElemType a[], int left, int right )
{
    int pivotloc;
    stackinit(); push2(left, right);
    while (!stackempty())
    {
        left = pop(); right = pop();
        if (left <= right)    continue;
        pivotloc = Partition (a, left, right ) ;
        if (pivotloc-left > right - pivotloc) { push2(left, pivotloc-1); push2(pivotloc+1,right); }
        else { push2(pivotloc+1,right); push2(left, pivotloc-1); } //子文件较小者压在栈顶先排序
    }
}

```

**性质：** 包含 $n$ 个元素的文件使用快速排序时，如果一趟划分得到的两个子文件中规模较小者先排好序，则栈中元素个数不会超过 $\log_2 n$ 。

## [算法的改进]

合理选择枢轴记录可改善性能。例如，

- 三者取中
- 随机产生

## [算法的稳定性]

是非稳定排序

反例  $[2, \underline{2}, 1]$ :  $\{1 \underline{2}\} \mathbf{2} \{ \}$

思考：分别用单链表、双向链表存储记录，  
快排算法如何实现？

## [三者取其中的算法]

```

Elemtype Media3(ElemType a[], int left, int right)
{
    int mid = (left + right)/2;
    if (a[left].key > a[mid].key)
        swap(&a[left], &a[mid]);
    if (a[left].key > a[right].key)
        swap(&a[right], &a[left]);
    if (a[mid].key > a[right].key)
        swap(&a[right], &a[mid]);
    swap (&a[mid], &a[right-1]); //枢纽在right-1
    return a[right-1].key; //返回枢纽值
} 枢纽

```

三者中小者放在a[left]，最大者放在a[right]，中值即枢纽放在a[right-1]，在分割阶段将分割指针i和j初始化到left+1, right-2，因为a[left]比枢纽小，所以它用作对j的警戒标记，不用担心j越界；由于i会停在等于枢纽的地方，把枢纽放在a[right-1]，它用作对i的警戒标记。

## [重写快速排序算法]

```

void QSort(elemtype a[],int left, int right)
{
    int i,j;
    ElemType pivot;
    pivot =Media3(a, left, right);
    i = left; j = right-1;
    for(;;) //一趟分割
    {
        while(a[++i].key < pivot.key) { } //先自加
        while(a[--j].key > pivot.key) { }
        if(i < j) swap(&a[i], &a[j]);
        else break;
    }
    swap(&a[i], &a[right-1]); //枢纽最终位置
    QSort(a, left, i-1);
    QSort(a, i+1, right);
}

```

[选择问题]-Selection problem: 在查找集 $S$ 中找出第 $k$ 个最小的元素。令 $|S_i|$ 为 $S_i$ 中元素个数。利用快速排序解决此问题的算法Qselect步骤:

1. 若 $|S|=1$ , 则 $k=1$ , 将 $S$ 中的元素作为答案返回。若数组规模较小, 则将 $S$ 采用直接插入排序等算法排好序, 并返回第 $k$ 个最小元素。
2. 选取一个枢纽 $v \in S$ 。
3. 将集合 $S - \{v\}$ 分割成 $S_1$ 和 $S_2$ 。
4. 若 $k \leq |S_1|$ , 则第 $k$ 个最小元素必在 $S_1$ 中, 则对 $S_1$ 递归求解; 若 $k = 1 + |S_1|$ , 则枢纽是第 $k$ 个最小元素, 返回枢纽。否则, 第 $k$ 个最小元素必在 $S_2$ 中, 它是 $S_2$ 中第 $(k - |S_1| - 1)$ 个最小元素, 对 $S_2$ 递归。



```

#define Cutoff (3)
void Qselect(ElemType a[], int k, int left, int right) //
{
    int i, j;
    ElemType Pivot;
    if (left + Cutoff <= right)
    {
        Pivot = Media3(a, left, right);
        i = Left; j = right - 1;
        for (;;)
        {
            while (a[++i] < Pivot) {}
            while (a[--j] < Pivot) {}
            if (i < j) swap(&a[i], &a[j]);
            else break;
        }
        swap (&a[i], &a[right - 1]);
        if (k <= i) Qselect(a, k, left, i-1);
        else if (k > i+1) Qselect(a, k, i+1, right);
    }
    else InsertSort(a, right-left+1)
}

```

### [驱动程序]

利用Qselect(a,k,0,n-1)调用程序时，数组a下标从0开始，则程序结束时a[k-1]为所求。

### [时间性能]

最坏情况下时间复杂度为： $O(n^2)$

平均情况下时间复杂度为： $O(n)$

平均每次划分为两个部分，因此，整个过程比较次数大约为：

$$n + n/2 + n/4 + \dots = 2n$$

## 9.4 选择排序

BUPT

### 1 简单选择排序(直接选择排序)

#### [算法步骤]

第1趟：从 $n$ 个记录中选关键字最小的记录，与第1个记录交换；

第2趟：从剩余的 $n-1$ 个记录中选关键字最小的记录，与第2个记录交换；

.....

第 $i$ 趟：从剩余的 $n-i+1$ 个记录中选关键字最小的记录，与第 $i$ 个记录交换；

.....

直到第 $n-1$ 趟执行完为止。

#### [算法描述]

## [简单选择排序]

```

void SelectSort (ElemType a[], int left, int right )
{
    for ( int i =left; i <= right - 1; ++i )
    {
        int j = SelectMinKey (a, i, right);
        if ( i != j ) swap(&a[i], &a[j]);
    }
} // SelectSort

```

## [主程序调用]

```

{
    .....
    SelectSort(a,1,n);
}

```

## [一趟选择]

```

int SelectMinKey (ElemType a[], int left, int right )
{
    int min = left;
    for (int i= left+1; i<= right; i++)
        if (a[i].key < a[min].key) min = i;
    return min; //返回小者的下标
}

```

[ 示例 ] (n=8)

|       |    |    |    |           |    |           |    |    |
|-------|----|----|----|-----------|----|-----------|----|----|
| <初态>  | 49 | 38 | 65 | 97        | 76 | <u>49</u> | 13 | 27 |
| <第1趟> | 13 | 38 | 65 | 97        | 76 | <u>49</u> | 49 | 27 |
| <第2趟> | 13 | 27 | 65 | 97        | 76 | <u>49</u> | 49 | 38 |
| <第3趟> | 13 | 27 | 38 | 97        | 76 | <u>49</u> | 49 | 65 |
| <第4趟> | 13 | 27 | 38 | <u>49</u> | 76 | 97        | 49 | 65 |
| <第5趟> | 13 | 27 | 38 | <u>49</u> | 49 | 97        | 76 | 65 |
| <第6趟> | 13 | 27 | 38 | <u>49</u> | 49 | 65        | 76 | 97 |
| <第7趟> | 13 | 27 | 38 | <u>49</u> | 49 | 65        | 76 | 97 |

(每趟排序使有序区增加一个记录)

不稳定排序

## [性能分析]

- ❖ 总的比较次数与记录排列的初始状态无关

$$C=(n-1)+(n-2)+\dots+2+1=n(n-1)/2$$

- ❖ 移动次数

初始记录逆序时:  $M_{\max} = 3(n-1)$

初始记录正序时:  $M_{\min} = 0$

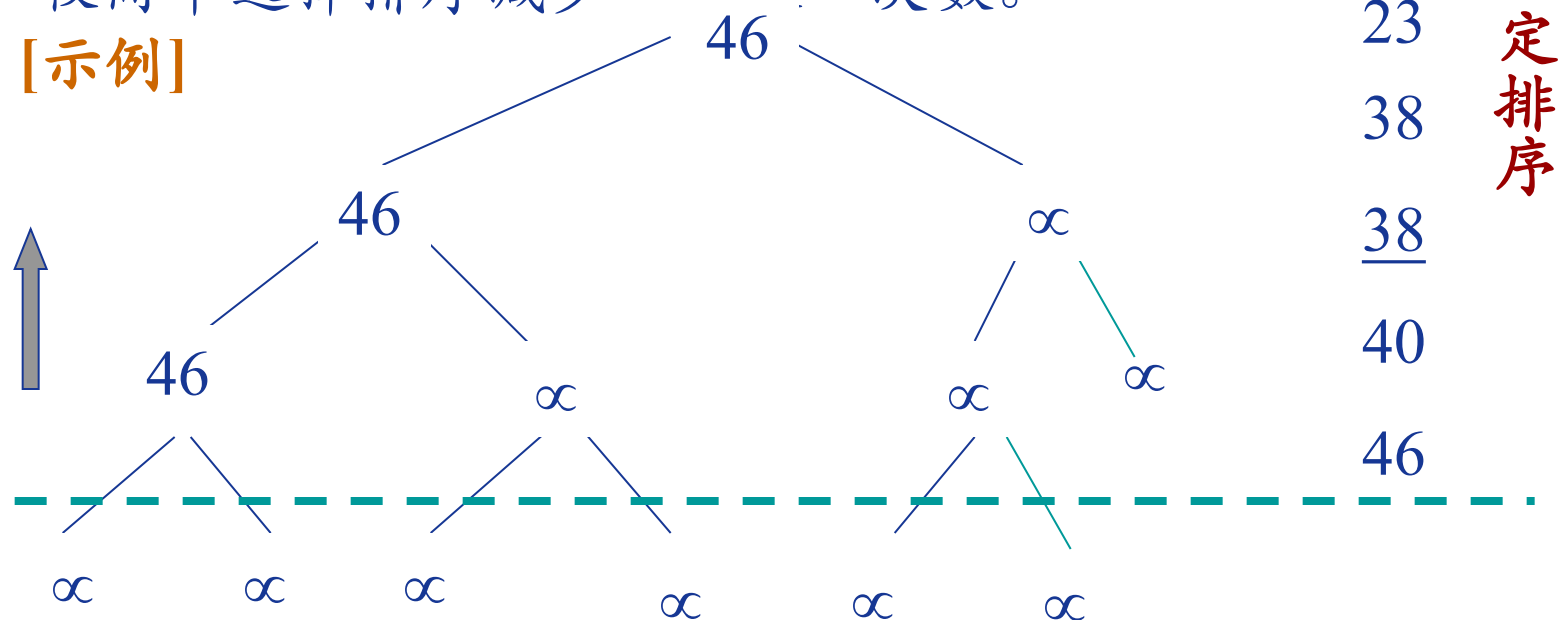
- ❖ 平均时间复杂度 $O(n^2)$

辅助空间复杂度 $O(1)$

## 锦标赛名次产生过程。

较简单选择排序减少“比较”次数。

### [ 示例 ]



# 稳定排序

**[性能]** 除第一次外，每次都走了树的一条分支，故其时间复杂度为  $O(n \log_2 n)$ 。

**[缺陷]** 辅助空间较多；需与“最大值”进行多余的比较

### 3 堆排序

#### [特点]

- 较直接选择排序减少重复比较;
- 较树形选择排序减少辅助存储空间(仅需一个)

#### [(二叉)堆的定义]

$n$ 个关键字的序列( $k_1, k_2, \dots, k_n$ )当且仅当满足下列条件之一:

- (1)  $K_i \leq K_{2i}$  且  $K_i \leq K_{2i+1}$     **小根(顶)堆**    ( $i=1, 2, \dots, \lfloor n/2 \rfloor$ )
- (2)  $K_i \geq K_{2i}$  且  $K_i \geq K_{2i+1}$     **大根(顶)堆**

则称该序列为一个堆。



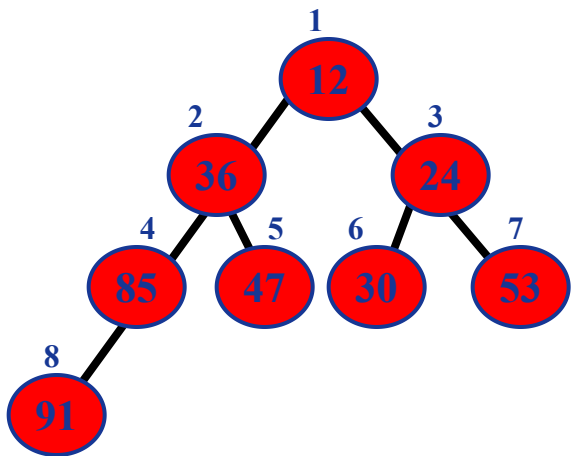
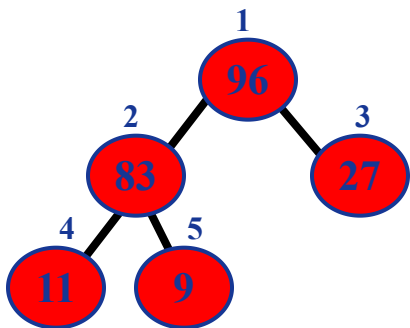
[示例]

{ 96, 83, 27, 11, 9 }

大根堆

{ 12, 36, 24, 85, 47, 30, 53, 91 }

小根堆



[堆对应于完全二叉树的存储结构]

|    |    |    |    |    |   |  |
|----|----|----|----|----|---|--|
| 96 | 83 | 27 | 38 | 11 | 9 |  |
| 1  | 2  | 3  | 4  | 5  | 6 |  |

## [堆排序基本思想]

将初始文件 $a[1..n]$ 建成一个大根堆(小根堆), 要排成正序选大根堆, 要排成逆序选小根堆;

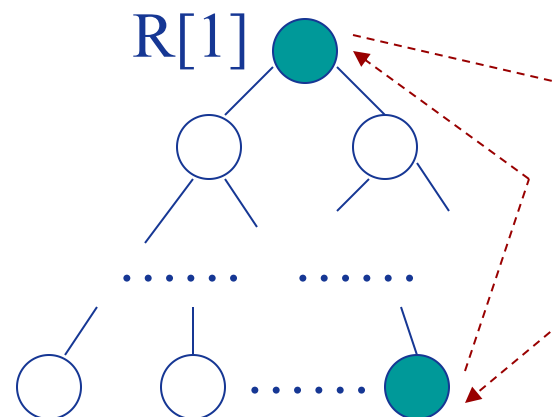
第1趟: 将关键字最大(最小)的记录(堆顶) $a[1]$ 与无序区的最后一个记录 $a[n]$ 交换; 再将新的无序区 $a[1..n-1]$ 调整为堆;

第2趟: 将 $a[1]$ 与 $a[n-1]$ 交换;  
再将新的无序区 $a[1..n-2]$ 调整为堆;

.....

第 $i$ 趟: 将 $a[1]$ 与 $a[n-i+1]$ 交换;  
再将新的无序区  
 $a[1..n-i]$ 调整为堆;

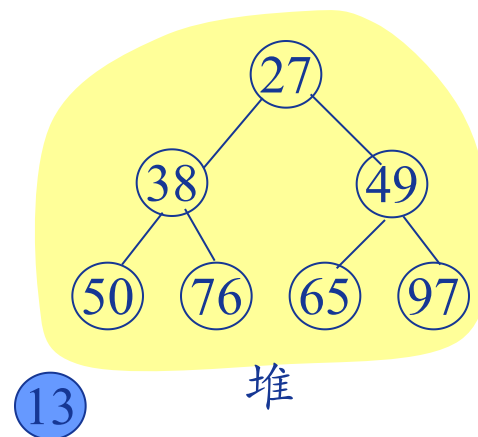
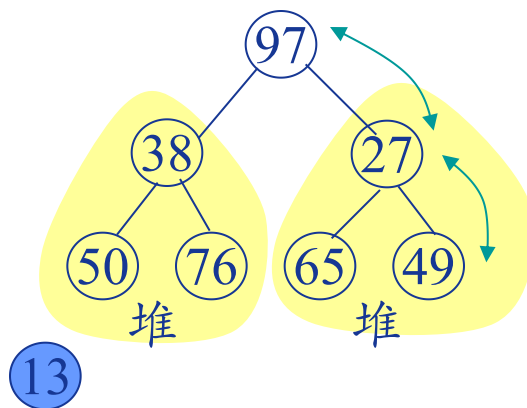
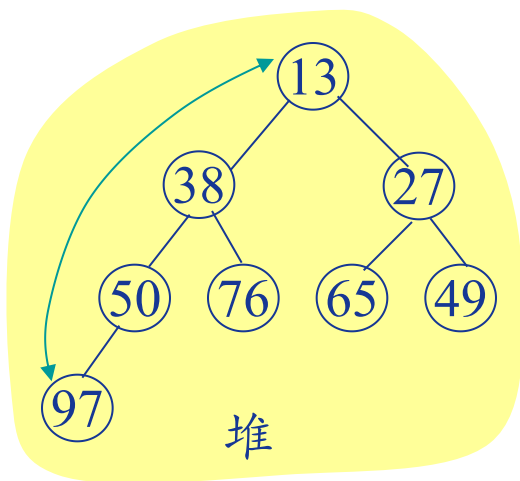
.....直到执行完第 $(n-1)$ 趟为止。



## [堆排序需解决的两个问题]

- ❖ 如何由一个无序序列建成一个大（小）根堆？
- ❖ 如何在输出堆顶元素之后，调整剩余元素，使之成为一个新的堆？

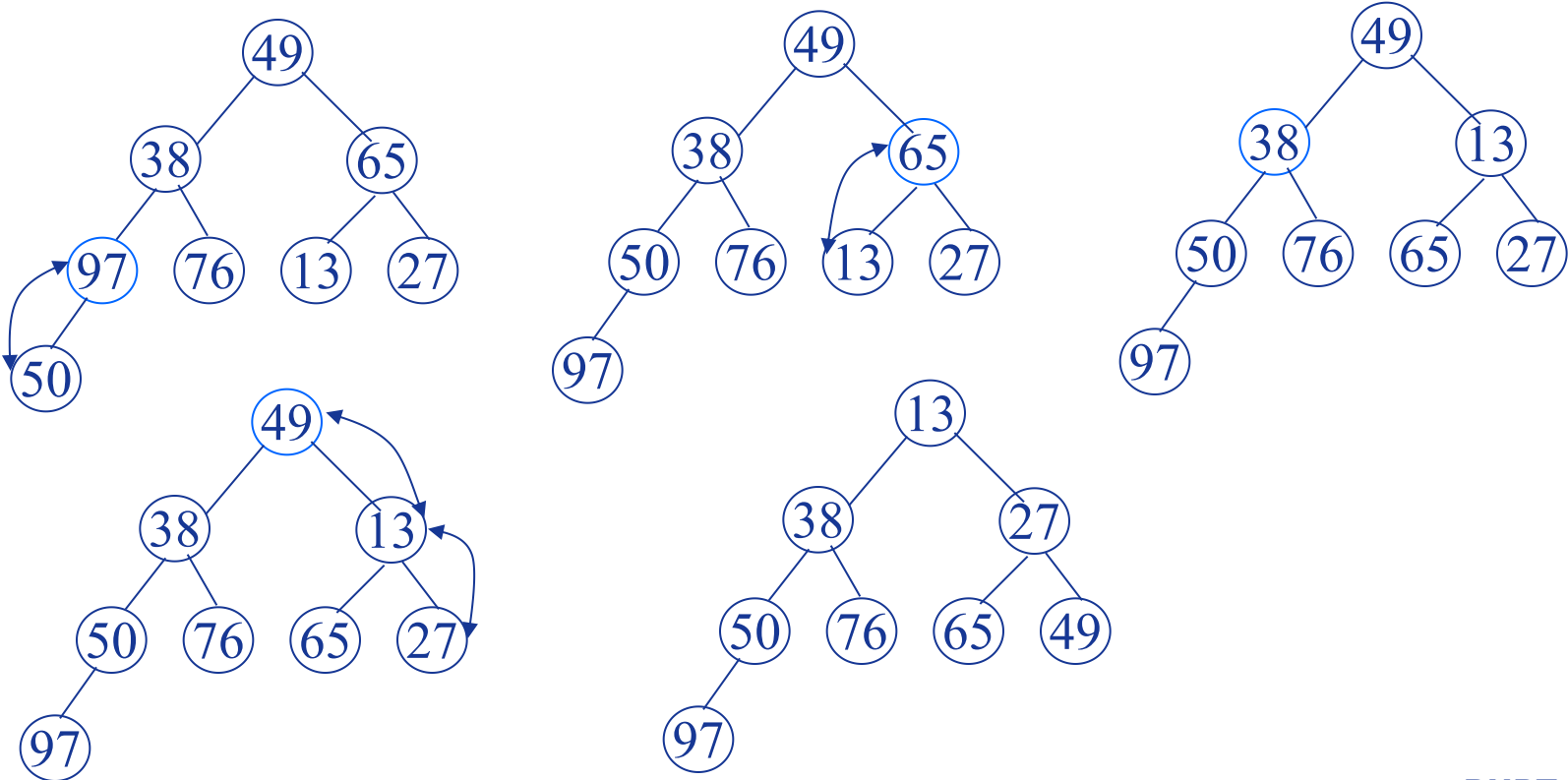
### 1) 调整堆的方法——筛选



2) 将初始记录序列R[1..n]建成大(小)根堆

依次把以 $i=\lfloor n/2 \rfloor, \lfloor n/2 \rfloor - 1, \lfloor n/2 \rfloor - 2, \dots, 1$ 为根的子树调整为小(大)根堆，则完成了整个建堆的过程。

(49, 38, 65, 97, 76, 13, 27, 50)



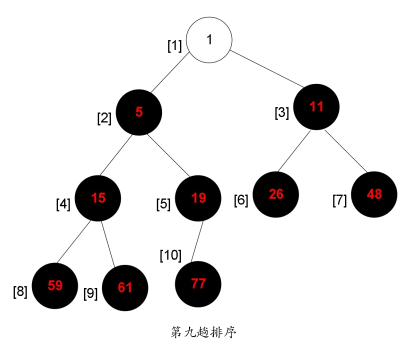
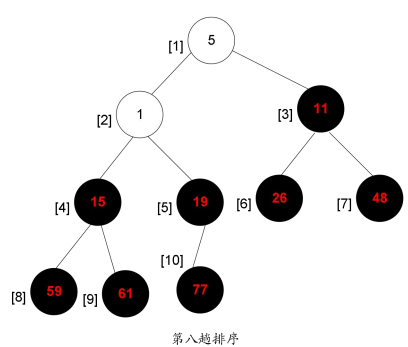
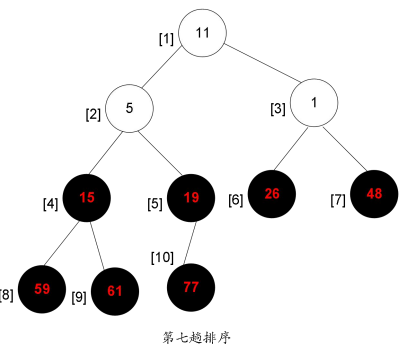
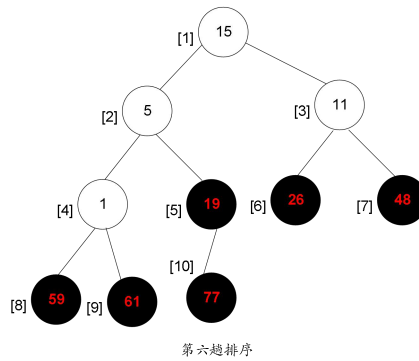
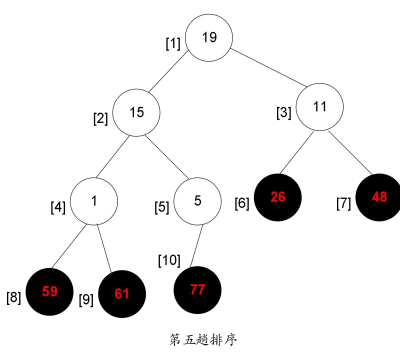
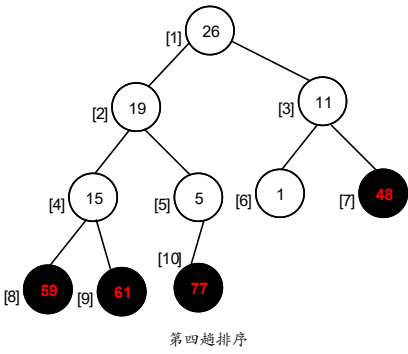
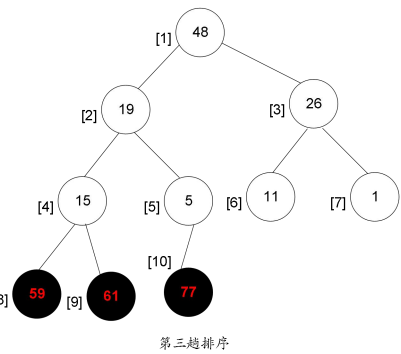
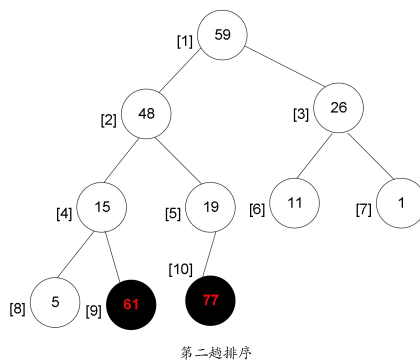
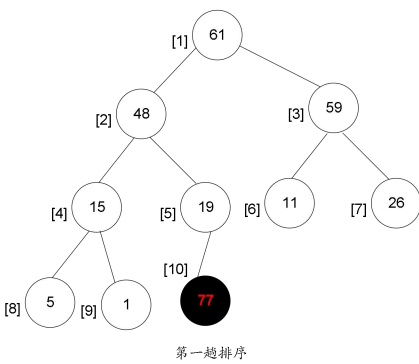
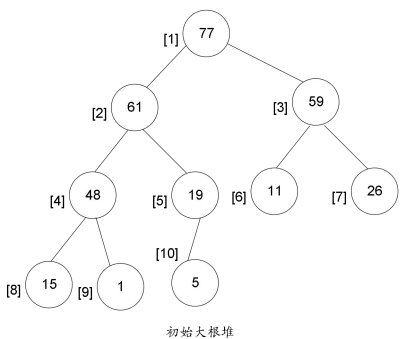
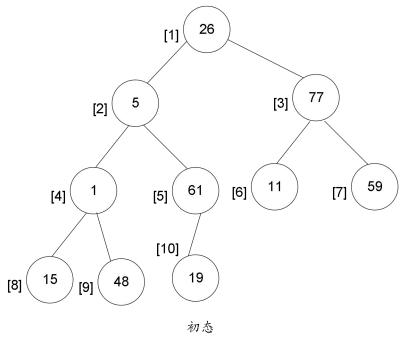
### [堆顶输出后调整为大根堆的算法]

```
void adjust(ElemType a[], int root, int end) {  
    int child = 2*root; //左子  
    KeyType rootkey = a[root].key; //缓存堆顶元素关键字  
    ElemType tmp = a[root]; //缓存堆顶元素  
    while (child <= end)  
    {  
        if (child < end && a[child].key < a[child+1].key) child++; //找左右子中大者  
        if (rootkey > a[child].key) //堆顶元素比最大的子元素大  
            break;  
        else  
        {  
            a[child/2] = a[child] //最大子上移覆盖双亲  
        }  
        child = child * 2;  
    }  
    a[child/2] = tmp; //原堆顶元素放在最终位置  
}
```

### [堆排序算法]

```
void HeapSort(ElemType a[], int n)
{
    int i, j;
    for (i = n/2; i > 0; i--)
        adjust(a, i, n);
    for (i = n-1; i > 0; i--)
    {
        swap(&a[1], &a[i+1]); //堆顶元素和最后一个交换位置
        adjust(a, 1, i);
    }
}
```

[算法实例]



## [HeapSort算法分析]

### 时间性能分析

假设  $2^{k-1} \leq n \leq 2^k$ ，则堆对应的完全二叉树有  $k$  层，第  $i$  层的结点数  $2^{i-1}$  (第  $k$  层可能不满)；

HeapSort 第一个 for 循环实现建立大根堆的过程，函数 HeapSort 将在每个有儿子的结点上调用一次 adjust 函数。因此建立大根堆的过程的时间开销为：

$$\sum_{i=1}^k 2^{i-1} (k-i) = \sum_{i=1}^k 2^{k-i-1} i \leq n * \sum_{i=1}^k \frac{2}{2^i} < 2n = O(n)$$

在第二个 for 循环中，函数 HeadSort 共调用 adjust 函数  $n-1$  次，且完全二叉树的最大深度为  $\lceil \log_2(n+1) \rceil$ ，因此该循环的时间复杂度为  $O(n \log_2 n)$ 。

因此总的的时间开销为：

$$O(n) + O(n \log_2 n) = O(n \log_2 n)。$$



## 空间复杂度

❖ 辅助空间： $O(1)$

## 稳定性

❖ 不稳定排序

思考：若有 $N$ 个元素已构成一个小根堆，那么如果增加一个元素为 $K_{n+1}$ ，如何在 $\log_2 n$ 的时间内将其重新调整为堆？

$K_1$ 到 $K_n$ 是堆，在 $K_{n+1}$ 加入后，将 $K_1..K_{n+1}$ 调成堆。

设 $c=n+1, f=\lfloor c/2 \rfloor$ ，若 $K_f \leq K_c$ ，则调整完成。

否则， $K_f$ 与 $K_c$ 交换；之后， $c=f, f=\lfloor c/2 \rfloor$ ，

继续比较，直到 $K_f \leq K_c$ ，或 $f=0$ ，即为根结点，调整结束。

# 9.5 归并排序

BUPT

## [归并的概念]

指将两个或两个以上的同序序列归并成一个序列的操作。

## 1 两路归并排序

### [算法思想]

第1趟：将待排序列 $R[1..n]$ 看作 $n$ 个长度为1的有序子序列，两两归并，得到 $\lceil n/2 \rceil$ 个长度为2的有序子序列(或最后一个子序列长度为1)；

第2趟：将上述 $\lceil n/2 \rceil$ 个有序子序列两两归并；

.....

直到合并成一个序列为止。

**[示例]** (n=7)

|       |      |      |      |           |      |           |      |
|-------|------|------|------|-----------|------|-----------|------|
| <初态>  | (49) | (38) | (65) | (97)      | (76) | (49)      | (13) |
| <第1趟> | (38  | 49)  | (65  | 97)       | (49  | 76)       | (13) |
| <第2趟> | (38  | 49   | 65   | 97)       | (13  | <u>49</u> | 76)  |
| <第3趟> | (13  | 38   | 49   | <u>49</u> | 65   | 76        | 97)  |

**[性能分析]**

- ❖ 任何情况时间复杂度 $O(n\log_2 n)$
- ❖ 空间复杂度 $O(n)$
- ❖ 很少用于内部排序

### [归并数组中两个已排序的子表]

//归并已排序的a[left..m] 和a[m+1..right], 排序结果存在sorted[left..right]

```
void merge(ElemType a[], ElemType sorted[], int left, int m, int right)
```

```
{
    int i, j, k, t;
    j = m+1; i = k = left;
    while(i<=m && j<=right)
    {
        if (a[i].key <= a[j].key)    sorted[k++] = a[i++];
        else    sorted[k++] = a[j++];
    }
    if (i<m)
        for (t=j; t<=right; t++)    sorted[k+t-j] = a[t]; //后半段还有剩余
    else
        for(t=i; t<=m; t++)    sorted[k+t-i] = a[t]; //前半段还有剩余
}
```

Merge函数分析: 每次while循环的操作都有一个记录加入结果表sorted中, sorted表大小为right-left+1; 因此while循环执行次数为right-left+1; merge函数的时间复杂度可表示为 $O(\text{right-left}+1)$ 。

## [一趟归并算法]

n是记录个数，length是子序列长度

```
merge_pass(ElemType a[], ElemType &sorted[], int n, int length)
```

```
{
```

```
    int i,j;
```

```
    for (i = 0; i <= n-2*length; i += 2*length) //取长度为length的两个归并段归并
```

```
        merge(a, sorted, i, i+length-1, i+2*length-1);
```

```
    if ( i+length < n ) //还剩两个归并段归并，一个长length，一个不足length
```

```
        merge(a, sorted, i, i+length-1, n-1);
```

```
    else
```

```
        for(j = i; j<n; j++) sorted[i] = a[j]; //只剩一个归并段，拷贝不归并
```

```
}
```

由merge函数的分析可知：merge\_pass函数的复杂度即所用归并段的长度之和： $O(n)$

## [归并的迭代算法]

```

对文件a[]进行归并 自底向上归并
mergesort(ElemType &a[], int n)
{
    int length = 1; //当前趟归并段长度
    ElemType extra[MAX_SIZE];
    while (length < n)
    {
        merge_pass(list, extra, n, length);
        length *= 2;
        merge_pass(extra, list, n, length);
        length *= 2;
    }
}

```

mergesort函数分析：归并排序是在输入记录上执行若干趟归并操作，其中第一趟归并长度为1的子序列，第二趟归并长度为2的子序列，第i趟归并长度为 $2^{i-1}$ 的子序列，因此总归并趟数为 $\lceil \log_2 n \rceil$ ，每一趟归并时间开销为 $O(n)$ ，因此总的的时间开销为： $O(n \log_2 n)$ 。

空间开销：归并要用到一个等长空间缓存中间结果  $O(n)$

[原位归并-重写merge函数]

//归并已排序的a[left..m] 和a[m+1..right], 排序结果还存在a[left..right]

```
void merge(ElemType a[], int left, int m, int right)
```

```
{
```

```
    ElemType aux;
```

```
    for (i = m+1; i > left; i--)
```

```
        aux[i-1] = a[i-1];
```

```
    for (j = m; j < right; j++)
```

```
        aux[r+m-j] = a[j+1];
```

```
    for (k = left; k < right; k++)
```

```
        if (aux[j].key < aux[i].key)
```

```
            a[k] = aux[j--];
```

```
        else
```

```
            a[k] = aux[j++];
```

```
}
```

原位归并的好处:

非原位归并每个循环都要处理a[left..m]和a[m+1..right] 是否越界。

原位归并: 原数组a前后两部分分别按照**倒序**复制 to 辅助数组aux, 这样最大元素成为监视哨, 减少边界处理。

[重写mergesort-自顶向下的归并]

```
void mergesort(ElemType a[], int left, int right)
{
    int m = (left + right)/2;
    if (right <= left) return;
    mergesort(a, left, m);
    mergesort(a, m+1, right);
    merge(a, left, m, right);
}
```

自顶向下归并的方法：将归并文件分成两部分，这两部分分别递归得到结果后，这两个结果合并成最终结果。



[重写mergesort-自底向上的归并]

```
#define min(A, B) (A < B) ? A : B
void mergesortBU(ElemType a[], int left, int right)
{
    int i, m; //m记录归并段长度，初始每归并段长1
    for (m = 1; m <= right-1; m = m+m)
        for (i = left; i <= right -m; i +=m+m)
            merge(a, i, i+m-1, min(i+m+m-1, right));
}
```

自底向下的归并：将文件中元素看成是大小为1的有序子表，进行1-1归并，产生大小为2的有序子表，进行2-2归并...依次类推，直到最后整个表有序。

在进行m-m归并后，下一趟m加倍。仅当文件大小是m的偶数倍时，最后子文件大小为m。因此每趟最后的归并是m-x归并，x小于等于m。

## [静态链表归并的递归算法]

在递归算法中将记录结构做一个小的修改，添加一个指针域。

```
typedef struct
```

```
{
    KeyType key;
    InfoType otherinfo;
    int next;
} ElemType;
```

时间复杂度:  $O(n\log_2 n)$

空间复杂度:  $O(n)$ ---键值不需要  
做移动, 但有额外指针空间

指针域next是整形的, 初始化 $a[i].next = -1$ 。  
函数rmerge实现递归算法, 返回一个指向已排序序列起始位置。

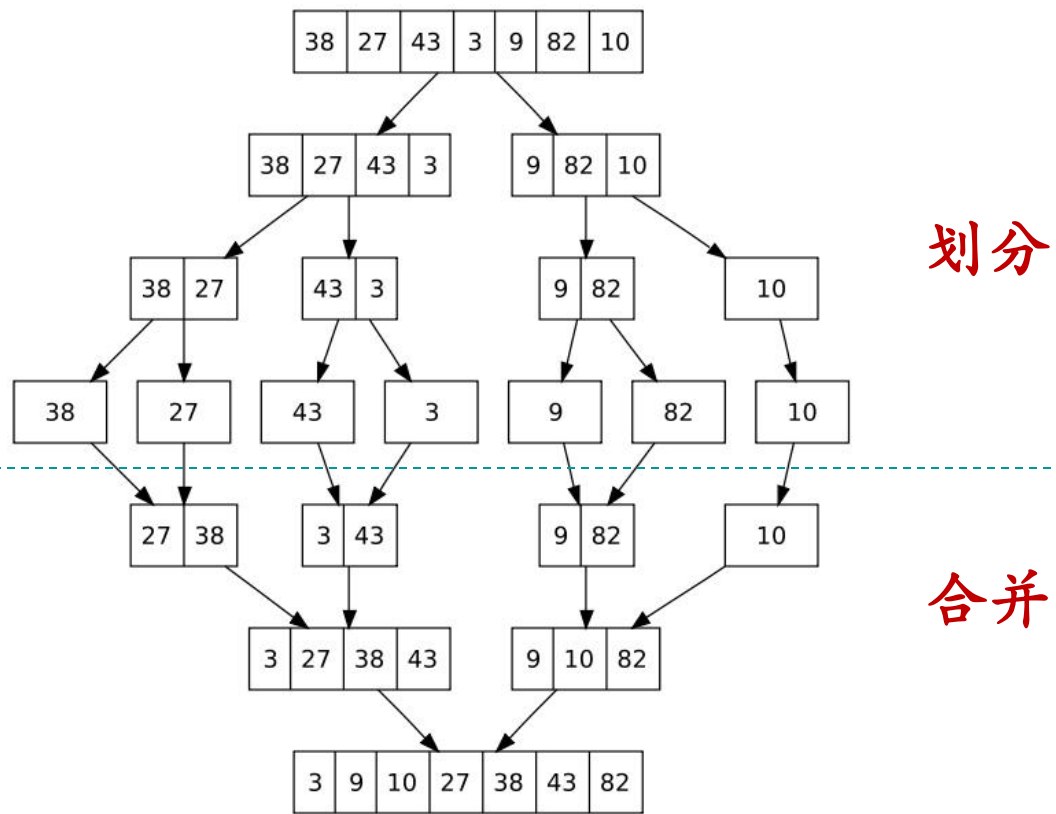
函数listmerge输入两个已排序链表first和second, 返回一个整数; 整数表示已归并排序单链表的起始位置。

```
int rmerge(ElemType &a[], int left, int right)
{//自顶向下划分 - 分治法
    int middle;
    if (left >= right) return left; //序列长1or 0有序, 退出
    else
    {
        middle = (left + right)/2;
        return listmerge(a, rmerge(a, left, middle),
                        rmerge(a, middle+1, right));
    }
}
```

```
listmerge(ElemType &a[], int first, int second)
{ //元素存放在a[0..n-1] a[n]头结点
    int start = n; //start指向排好序链表的首元素
    while (first != -1 && second != -1)
        if (a[first].key <= a[second].key)
        {
            a[start].next = first;
            start = first;
            first = a[first].next;
        }
        //first指向第一归并段表下一个待插入元素
    else
    {
        a[start].next = second;
        start = second;
        second = a[second].next;
    }
    if (first == -1) a[start].next = second;
    else a[start].next = first;
    return a[start].next;
}
```

[静态单链表归并的实例]

假设输入序列为(38,27,43,3,8,82,10)每次递归调用rmerge时都将当前下标从left到right划分成两个子序列（**分治法**），两个子序列下标范围L left..(left+right)/2 ⌊ 和L (left+right)/2 ⌊ +1..right。然后分别对这些子序列进行递归，直到子序列长度为0或者1，再归并其结果。



## [单链表的归并]

[结点结构]

```
struct node
```

```
{
```

```
    struct node *next;
```

```
    elemtype data;
```

```
};
```

```
typedef struct node Node;
```

```
typedef struct node *Position;
```

```
typedef struct node *List;
```

[两个有序单链表的归并]

```
List merge(List a, list b)
```

```
{//不带头结点
```

```
    Node head;
```

```
    List c = &head;
```

```
    while ((a != NULL) && (b != NULL))
```

```
    if (a->data.key < b->data.key)
```

```
    {
```

```
        c->next = a; c=a; a = a->next;
```

```
    }
```

```
    else
```

```
    {
```

```
        c->next = b; c = b; b = b->next;
```

```
    }
```

```
    c->next = (a == NULL) ? b: a;
```

```
    return head.next;
```

```
}
```

[自顶向下单链表归并排序]

List mergesort(List c)//不带头结点

```
{
    List a, b;
    if (c == NULL || c->next == NULL )
        return c;
    a = c; b = c->next;
    while ((b) && (b->next))
    {
        c = c->next;
        b = b->next->next;
    }
    b = c->next; c->next = NULL;
    return merge(mergesort(a), mergesort(b));
}
```

1. 程序把c指向的表分为两部分：一部分由a指向，一部分由b指向；
2. 分别对a和b递归排序；
3. 使用merge得到最终结果。

[自底向上单链表归并排序]

```
List mergesort(List t) {
    List u;
    for(Qinit(); t; t = u) {
        u = t->next;
        t->next = NULL;
        Qput(t); //入队列
    }
    t = Qget(); //出队列
    while (!Qempty()) {
        Qput(t);
        t = merge(Qget(), Qget());
    }
    return t;
}
```

程序使用队列Q来实现自底向上的归并，队列元素为有序链表，元素的初始表长为1。

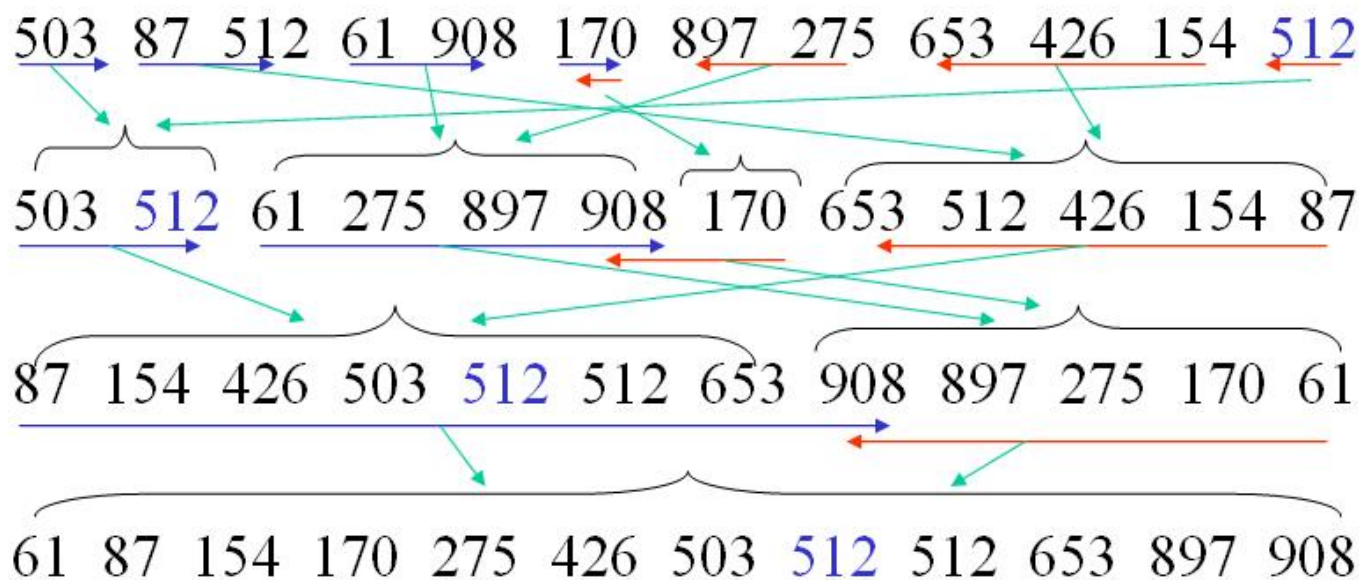
程序从队列中取出两个有序链表，将它们归并完成后，归并结果放回队列中。继续这一过程，直到队列中只有一个元素。每一趟有序表的长度加倍。

### 3 自然两路归并排序

#### [特点]

以游程(自然的有序段)作为子序列进行归并，可以比直接两路归并更有效。

#### [示例]



不稳定排序

## 9.7 基数排序

BUPT

**[特点]** 通过“分配”和“收集”过程来实现排序，时间复杂度可以突破基于关键字比较一类方法的下界 $O(n\lg n)$ ，达到 $O(n)$ 。

**[方法]** 借助多关键字排序的思想对单关键字排序

**[多关键字排序示例]** 扑克牌排序(点数+花色)

双关键字 花色：♣ < ♦ < ♥ < ♠

点数：2 < 3 < ... < Q < K < A

“花色”地位高于“面值”

**最高位优先法：**按花色分成4堆；

**(MSD法)** 对每一堆：按面值分堆；

从小到大排序；

按花色从小到大排序



**最低位优先法：** 按点数大小分成13堆；  
**(LSD法)** 按点数从小到大收集起来；  
再按花色分成四堆；  
按花色从小到大收集起来

### [MSD与LSD不同特点]

- ❖ 按MSD排序，必须将序列逐层分割成若干子序列，然后对各子序列分别排序
- ❖ 按LSD排序，不必分成子序列，对每个关键字都是整个序列参加排序；并且可不通过关键字比较，而通过若干次分配与收集实现排序

## [基数排序算法思想] 借鉴LSD法

设参加排序的序列为  $K = \{K_1, K_2, \dots, K_n\}$ ,

其中  $K_i$  是  $d$  位  $rd$  进制的数,  $rd$  称为 **基数**;

$d$  由所有元素中最长的一个元素的位数计量,

$$K_i = K_i^1 K_i^2 \dots K_i^d$$

高
低

从低位到高位依次对  $K_j (j=d, d-1, \dots, 1)$  根据基数分配, 再按基数递增序收集, 则可得有序序列。

### [例]

(1)  $K = \{3621 \text{ } 0724 \text{ } 8385 \text{ } 0075 \text{ } 0514 \text{ } 7368 \text{ } 0008 \}$

$rd=10, d=4$

(2)  $K = \{\text{Zhang Wang} \blacksquare \text{ Li} \blacksquare \blacksquare \text{ Zhao} \blacksquare\}$

$rd=27, d=5$

**rd=10**  
**d=3**

$$\begin{array}{cccc} \downarrow 241 & 363 & \downarrow 5 & \downarrow 477 \\ 81 & & 5 & 467 \end{array}$$

收集 {241 81 363 5 5 477 467}

$$\begin{array}{ccccccc} \downarrow 5 & & 241 & & \downarrow 363 & 477 & 81 \\ & & & & 467 & & \end{array}$$

收集  $\{5 \quad \underline{5} \quad 241 \quad 363 \quad 467 \quad 477 \quad 81\}$

$$\begin{array}{r} 5 \\ \downarrow \\ \underline{5} \\ \downarrow \\ 81 \end{array} \quad \begin{array}{r} 241 \quad 363 \end{array} \quad \begin{array}{r} 467 \\ \downarrow \\ 477 \end{array}$$

收集 {5 5 81 241 363 467 477}

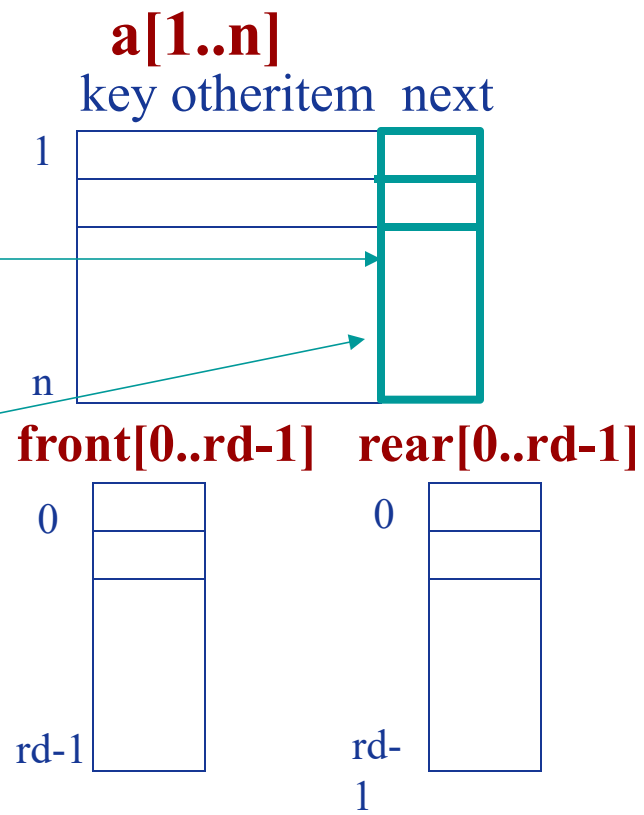
## 稳定排序 SCST

[数据结构]: 一般采用链式结构存储

- ❖ 主: ElemType a[1..n]
  - 数据域( key + otheritem ): 记录
  - 指针域( next ): 下一个记录的序号

```
typedef struct {
    KeyType key;
    InfoType otherinfo;
    int next;
} ElemType;
```

和链表归并的数据结构一样



- ❖ 辅: front[0..rd-1] 各链式队列头指针  
rear[0..rd-1] 各链式队列尾指针

```

int RadixSort (ElemType &a[], int d, int rd, int n)//对a[1..n]按d位rd进制进行基数排序，函数返回链表头
{ //说明：程序中使用的函数int digit(a[i],j,rd)，其返回值为rd进制下，a[i]的key的第j位，值0..rd-1
    int front[rd], rear[rd]; //取值范围0..rd-1
    int i, bin, current, first, last;
    first = 1; //链表头赋初值
    for(i = 1; i < n; i++) a[i].next = i+1; //初始化next指针域
    a[n].next = 0; //表示第n项后继为空;
    for(i = d-1; i >= 0; i--) { //按第i位排序
        for (bin = 0; bin < rd; bin++) front[bin] = 0; //队头指针初始化
        for (current = first; current; current = a[current].next) { //每次从链表中取一个元素分配
            bin = digit(a[current],j,rd); //返回值0..rd-1中的一个
            if (front[bin] == 0) front[bin] = current;
            else a[rear[bin]].next = current; //改写当前分配元素在队列中的前驱元素的后继指针，使得其后继指向当前元素。
            rear[bin] = current; //分配当前元素到队列
        }
        for (bin = 0; !front[bin]; bin++); //找到第一个非空队列开始准备收集
        first = front[bin]; last = rear[bin];
        for (bin++; bin < rd; bin++) //收集
            if (front[bin]) {
                a[last].next = front[bin]; //两个队列首尾相连;
                last = rear[bin];
            }
        a[last].next = 0; //链表表尾next置空，收集完成，也标志着一趟排序结束。
    }
    return first; //返回链表头指针
}

```

#### [性能分析]

- ❖ 每一趟：分配 **$O(n)$**  收集 **$O(rd)$**   
 d趟总计： **$O(d(n+rd)) \Rightarrow O(n)$**  通常d, rd均为常数
- ❖ 辅助空间
  - n个指针域空间
  - 队头指针数组front[0..rd-1]和队尾指针数组rear[0..rd-1]
 辅助空间复杂度： **$O(rd+n)$**

## 9.7 各种内部排序算法的比较

BUPT

| 排序方法 | 最好时间           | 平均时间           | 最坏时间           | 辅助空间          | 稳定性 |
|------|----------------|----------------|----------------|---------------|-----|
| 直接插入 | $O(n)$         | $O(n^2)$       | $O(n^2)$       | $O(1)$        | √   |
| 希尔   |                | $O(n^{1.3})$   |                | $O(1)$        | ×   |
| 冒泡   | $O(n)$         | $O(n^2)$       | $O(n^2)$       | $O(1)$        | √   |
| 快速   | $O(n\log_2 n)$ | $O(n\log_2 n)$ | $O(n^2)$       | $O(\log_2 n)$ | ×   |
| 简单选择 | $O(n^2)$       | $O(n^2)$       | $O(n^2)$       | $O(1)$        | ×   |
| 堆    | $O(n\log_2 n)$ | $O(n\log_2 n)$ | $O(n\log_2 n)$ | $O(1)$        | ×   |
| 归并   | $O(n\log_2 n)$ | $O(n\log_2 n)$ | $O(n\log_2 n)$ | $O(n)$        | √   |
| 基数   | $O(d(rd+n))$   | $O(d(rd+n))$   | $O(d(rd+n))$   | $O(rd+n)$     | √   |

- ❖ 按平均时间排序方法分为四类  
 $O(n^2)$ 、 $O(n\lg n)$ 、 $O(n^{1+\varepsilon})$ 、 $O(n)$
- ❖ 快速排序是目前基于比较的内部排序中最好的方法
- ❖ 关键字随机分布时，快速排序的平均时间最短，堆排序次之，但后者所需的辅助空间少
- ❖ 当 $n$ 较小时如( $n < 50$ )，可采用直接插入或简单选择排序，前者是稳定排序，但后者通常记录移动次数少于前者
- ❖ 当 $n$ 较大时，应采用时间复杂度为 $O(n\lg n)$ 的排序方法(主要为快速排序和堆排序)或者基数排序的方法，但后者对关键字的结构有一定要求
- ❖ 当 $n$ 较大时，为避免顺序存储时大量移动记录的时间开销，可考虑用链表作为存储结构（如插入排序、归并排序、基数排序）

- ❖ 快速排序和堆排序难于在链表上实现，可以采用地址排序的方法，之后再按辅助表的次序重排各记录
- ❖ 文件初态基本按正序排列时，应选用直接插入、冒泡或随机的快速排序
- ❖ 选择排序方法应综合考虑各种因素

讨论：假设有  $n$  个值不同的元素存于顺序结构中，要求不经排序选出前  $k$  ( $k \ll n$ ) 个最小元素，问哪些方法可用，哪些方法比较次数最少？

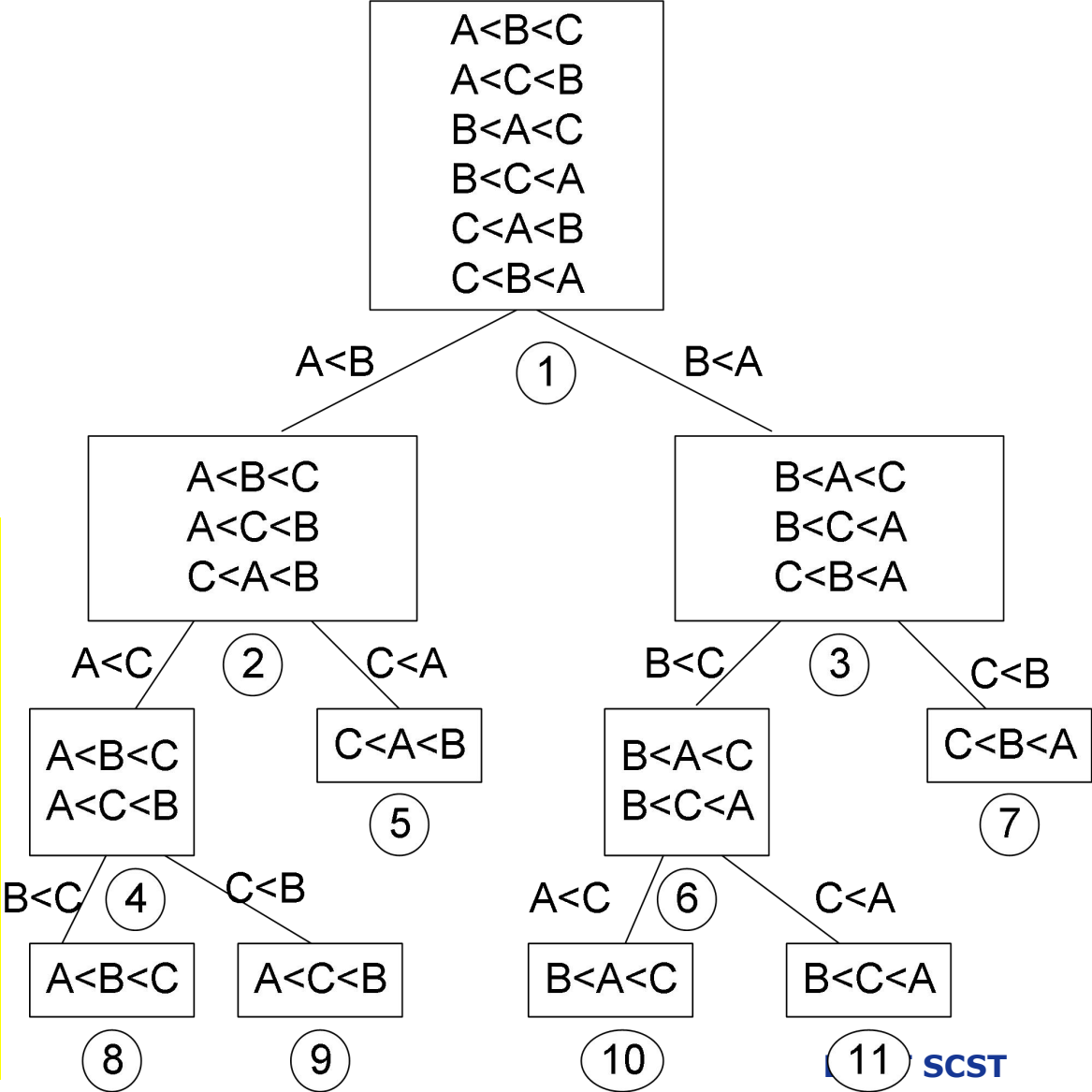
- 选择排序或冒泡排序： $k$ 趟（数据比较次数约为  $kn$ 次）；
- 快速排序：每次仅对第一个子序列划分，直至子序列长度小于等于  $k$ ；长度不足  $k$ ，则再对其后的子序列划分出补足的长度即可（平均对数据比较  $2n$ 次）
- 堆排序：先建小根堆， $k-1$ 次堆调整（数据比较次数约为  $4n + (k-1)\log_2 n$ ）



[排序的一般下界]

决策树(Decision Tree)  
是用于证明下界的抽象概念，它是一棵二叉树，结点表示的是在元素之间一组可能的排序，它与已经进行的排序一致。

ABC三元素排序决策树  
初始状态在根结点，表示未进行任何比较，第一次比较A和B，比较结果呈现两种可能；若A<B，则三种可能留下，算法到达结点2，接下来比较A和C，若A<C，算法进入结点5.....



## [排序的一般下界]

定理1：只使用元素间比较的任何排序算法在最坏情况下至少需要进行  $\lceil \log_2(n!) \rceil$  次比较。

证明：对  $n$  个元素的决策树必然有  $n!$  个叶子结点，具有  $n!$  个叶子的二叉决策树高度至少是  $\lceil \log_2(n!) \rceil$ ；而比较次数最多为决策树的高度，因此在最坏情况下至少需要进行  $\lceil \log_2(n!) \rceil$  次比较。

定理2：只使用元素间比较的任何排序算法需要进行  $\Omega(n \log_2 n)$  次比较。

证明：由定理1知，至少需要  $\log_2(n!)$  次比较。

$$\begin{aligned}
 \log_2(n!) &= \log_2(n(n-1)(n-2)\dots(2)(1)) \\
 &= \log_2 n + \log_2(n-1) + \log_2(n-2) + \dots + \log_2(2) + \log_2(1) \\
 &\geq \log_2 n + \log_2(n-1) + \log_2(n-2) + \dots + \log_2(n/2) \\
 &\geq (n/2) * \log_2(n/2) \\
 &\geq (n/2) * \log_2(n) - n/2 \\
 &= \Omega(n \log_2 n)
 \end{aligned}$$

# 作业

BUPT

1. 全国有10000人参加物理竞赛，只录取成绩优异的前10名，不需排出名次。问此种情况下，用何种排序方法速度最快？为什么？
2. 若待排序列用带头结点的单链表存储，试给出简单选择排序算法。

```
typedef struct node {  
    int    key;  
    InfoType otherinfo;  
    struct node  *next;  
} node, *pointer;  
void selectsort(pointer &h)//h为头指针
```

- 3.请编写出算法，借助于快速排序的算法思想，在一组无序的记录中查找给定关键字值等于k的记录是记录序列中第几大的数。设此组记录存于数组r[low..high]中，若查找成功，则返回该记录是r数组中第几大的数，否则返回0。

```
int Index (ElemType r[], int low, int high, int k)
```

```
// ElemType结构参见PPT P6
```

- 4.以关键码序列(503, 087, 512, 061, 908, 170, 897, 275, 653, 426)为例，手工执行以下排序算法，写出每一趟排序结束时的关键码状态：

- (1) 直接插入排序
- (2) 希尔排序(d[1]=5, d[2]=3, d[3]=1)
- (3) 快速排序(第一个记录为基准记录)
- (4) 堆排序
- (5) 归并排序
- (6) 基数排序